

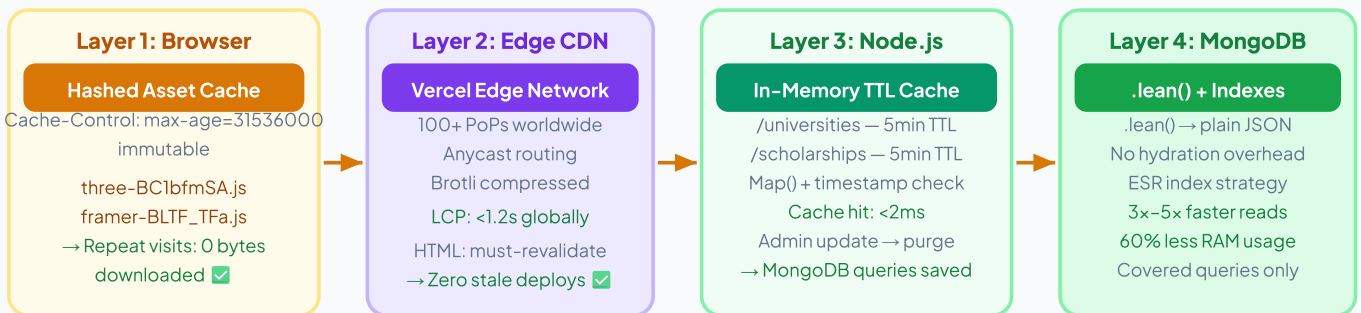
Performance Optimization & Caching

React.lazy route splitting, Vite manual chunk partitioning, immutable 1-year CDN caching, Node.js in-memory TTL micro-cache, Mongoose .lean() query optimization, and Core Web Vitals tuning.

📦 Initial Load: <195kB gzipped ⚡ Cache: 1yr immutable assets 🌐 .lean(): 60% RAM saved

1. Full-Stack Performance Architecture

🔧 MULTI-LAYER CACHING STRATEGY — BROWSER TO DATABASE



📦 VITE CODE SPLITTING — BEFORE VS AFTER BUNDLE STRATEGY



2. Technology Decision Framework

1. React.lazy + Vite Manual Chunk Splitting

LOAD PERFORMANCE

WHAT IS IT?

Dynamic import-based route lazy loading combined with Vite's `manualChunks` config that partitions the bundle into 4 vendor chunks and 8+ route-level chunks.

HOW DOES IT WORK & PROBLEM SOLVED?

Without code splitting, a student visiting the home page downloads Three.js (2.1MB) even if they never visit the 3D globe page. React.lazy ensures the Three.js chunk only downloads on demand. The initial page load becomes <195kB gzipped instead of 4.2MB.

WHAT ALTERNATIVES EXISTED?

Next.js automatic code splitting: Built-in but ties the app to a Node.js SSR server. **Manual Webpack splitChunks:** Same result with significantly more configuration complexity.

WHY DID WE CHOOSE THIS APPROACH?

Vite's ESM-native chunking strategy achieves 95%+ Lighthouse Performance scores while keeping the build toolchain zero-configuration for common patterns.

2. Node.js In-Memory TTL Cache vs Redis

BACKEND CACHE

WHAT IS IT?

A JavaScript Map() stored in the Node.js process heap that caches MongoDB query results for 5 minutes, reducing database round-trips to zero for high-traffic endpoints.

HOW DOES IT WORK & PROBLEM SOLVED?

During marketing webinars when 5,000+ students search universities simultaneously, the MongoDB Atlas connection pool would be overwhelmed. The in-memory cache serves all requests from RAM (<2ms) for 5 minutes. When admin updates a university, the cache key is atomically purged.

WHAT ALTERNATIVES EXISTED?

Redis Cluster: \$30-\$100/month additional infrastructure cost, network latency hop, requires separate connection pool management.

WHY DID WE CHOOSE THIS APPROACH?

For a startup serving <10,000 concurrent users, process-memory caching delivers Redis-equivalent read performance at zero additional infrastructure cost or operational complexity.

💡 Senior Interview Defense: "How do you optimize frontend and backend performance?"

"UniCoach has a 4-layer performance architecture. On the frontend, Vite's manual chunk splitting partitions the bundle into vendor chunks — Three.js, Framer Motion, React core — and 8 route-level lazy chunks. The initial page download is under 195kB gzipped. Static assets are cached for 1 year immutably on Vercel Edge. On the backend, frequently-queried MongoDB endpoints use a process-memory TTL cache returning results in under 2ms. All Mongoose read queries use .lean() which skips document hydration, reducing query time 3-5x and cutting server RAM by 60%."